

End-to-End SQL Practice for MySQL Workbench

Covers: CREATE DATABASE, ORDER BY, filtering, LIMIT, GROUP BY, math operations, joins, UPDATE/DELETE, stored procedures, ETL, and statistics.

End-to-End SQL Practice (MySQL Workbench)

1) Create Database + Use It

```
-----  
CREATE DATABASE sql_practice_demo;  
USE sql_practice_demo;
```

2) Create Tables

```
-----  
CREATE TABLE departments (  
    department_id INT PRIMARY KEY AUTO_INCREMENT,  
    department_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department_id INT,  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    FOREIGN KEY (department_id)  
        REFERENCES departments(department_id)  
);
```

```
CREATE TABLE sales (  
    sale_id INT PRIMARY KEY AUTO_INCREMENT,  
    employee_id INT,  
    sale_amount DECIMAL(10,2),  
    sale_date DATE,  
    FOREIGN KEY (employee_id)  
        REFERENCES employees(employee_id)  
);
```

3) Insert Sample Data

```
-----  
INSERT INTO departments (department_name)  
VALUES ('Engineering'), ('HR'), ('Finance');
```

```
INSERT INTO employees  
(first_name, last_name, department_id, salary, hire_date)  
VALUES  
( 'Amit', 'Yadav', 1, 90000, '2024-01-10'),  
( 'Sara', 'Khan', 2, 60000, '2023-11-05'),  
( 'Ravi', 'Sharma', 1, 85000, '2024-03-15'),  
( 'Neha', 'Gupta', 3, 75000, '2022-09-20');
```

```
INSERT INTO sales  
(employee_id, sale_amount, sale_date)  
VALUES  
(1, 5000, '2026-05-01'),  
(1, 8000, '2026-05-10'),  
(3, 4500, '2026-05-08'),  
(4, 7000, '2026-05-12');
```

4) Sorting Data using ORDER BY

```
-----  
SELECT * FROM employees  
ORDER BY salary DESC;
```

5) Filtering using Comparison Operators

```
-----  
SELECT * FROM employees  
WHERE salary > 70000;
```

```
SELECT * FROM employees  
WHERE hire_date >= '2024-01-01';
```

6) Limiting Results using LIMIT

```
-----  
SELECT * FROM employees
```

```
ORDER BY salary DESC
LIMIT 2;
```

7) GROUP BY Clause

```
-----
SELECT department_id,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

8) Mathematical Operations

```
-----
SELECT first_name,
       salary,
       salary * 0.10 AS bonus,
       salary + (salary * 0.10) AS total_salary
FROM employees;
```

9) Order of Operations

```
-----
SELECT first_name,
       salary,
       (salary + 5000) * 1.10 AS revised_salary
FROM employees;
```

10) SQL Joins (INNER / LEFT / RIGHT)

```
-----
-- INNER JOIN
SELECT e.first_name,
       d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

```
-- LEFT JOIN
SELECT e.first_name,
       s.sale_amount
FROM employees e
LEFT JOIN sales s
ON e.employee_id = s.employee_id;
```

```
-- RIGHT JOIN
SELECT e.first_name,
       s.sale_amount
FROM employees e
RIGHT JOIN sales s
ON e.employee_id = s.employee_id;
```

11) Updating and Deleting Data

```
-----
UPDATE employees
SET salary = salary + 5000
WHERE employee_id = 2;
```

```
DELETE FROM sales
WHERE sale_amount < 5000;
```

12) Stored Procedure

```
-----
DELIMITER //

CREATE PROCEDURE GetEmployeesByDepartment(
    IN dept_id INT
)
BEGIN
    SELECT *
    FROM employees
    WHERE department_id = dept_id;
END //
```

```
DELIMITER ;
```

```
CALL GetEmployeesByDepartment(1);
```

13) ETL using SQL

```
CREATE TABLE employee_backup AS  
SELECT *  
FROM employees;
```

14) Simple Statistics

```
-----  
SELECT  
    MIN(salary) AS min_salary,  
    MAX(salary) AS max_salary,  
    AVG(salary) AS avg_salary,  
    COUNT(*) AS total_employees  
FROM employees;
```

15) Cleanup (Optional)

```
-----  
DROP TABLE sales;  
DROP TABLE employees;  
DROP TABLE departments;  
DROP DATABASE sql_practice_demo;
```